School of Earth, Ocean & Climate Sciences

# ASSIGNMENT 3 - FORTRAN

COURSE CL5P252

NAME OF THE COURSE – NUMERICAL MODELLING AND DIAGNOSTICS LAB

NAME – AAROKSH CHAUHAN

PROFESSOR NAME- DR. SANDEEP K.K

DATE OF LABORATORY – 23 MARCH 2026

DATE OF SUBMISSION – 28 MARCH 2026

# FORTRAN PROGRAMMING

## Q.1) Write a Fortran program to estimate the Mean, Standard Deviation.

Solution – For finding out the mean of one dimensional array in fortran , we have used the following code :

```
program Lab_23mar_26
    implicit none

    real, dimension(10) :: x
    real :: total_sum,mean_val
    integer :: n

    x = (/ 13,55,84,36,28,58,29,12,25,97 /)
    total_sum = sum(x)
    n = 10
    mean_val = total_sum/n


    print *,"Mean value: ", mean_val
end program Lab_23mar_26

"mean.f90" 16L, 293B                                    1,1          All
```

In this code , we have took into consideration an one dimensional array x = (13,55,84,36,28,58,29,12,25,97) . So basically we have firstly took the sum of these values , then divided it with the length of the array n (here, 10) . “print *, helps us in printing out the mean values which is coming out to be

```
user@DESKTOP-Q2I4J5P:~$ ./a.out
 Mean value:     43.7000008
```

And , for finding out the standard deviation , we have used the code :

```
user@DESKTOP-Q2I4J5P: ~

program Mean_StdDev
    implicit none
    integer, parameter :: n = 10
    real, dimension(n) :: data
    real :: mean, stddev
    integer :: i

    data = (/ 13, 55, 84, 36, 28, 58, 29, 12, 25, 97 /)


    mean = sum(data) / n


    stddev = sqrt(sum((data - mean)**2) / (n - 1))

    print *, "Standard Deviation: ", stddev

end program Mean_StdDev
~
~
~
~
~
~
~
~
~
~
~
"stddev.f90" 18L, 355B                                    18,5          All
```

As we can see in this code , we have assumed another one dimensional array named data = (13,55,84,36,28,58,29,12,25,97) . To find its standard deviation , we have simply calculated its mean using the formula [sum of data / length of data], then doing the difference between the actual data and its mean values , summing up these values and taking square of it .

At last , divide the obtained number with the (length of data – 1) , here length of data is denoted by n . Hence , the obtained result is :

```
user@DESKTOP-Q2I4J5P:~$ ./a.out
 Standard Deviation:     29.0900898
```

**Q.2 ) Write a Fortran program to estimate the Correlation coefficient between two one-dimensional arrays.**

Solution – To compute Correlation coefficient between two one dimensional arrays , we have used the following code for determining the correlation coefficient :

```
user@DESKTOP-Q2I4J5P: ~

program Lab_23mar_26
    implicit none

    real, dimension(10) :: x, y
    real :: mean_x, mean_y, corr , numerator , denom_x , denom_y
    integer :: n

    x = (/ 13,55,84,36,28,58,29,12,25,97 /)
    y = (/ 71,9,26,70,19,50,30,23,22,44 /)
    n = 10

    mean_x = sum(x) / n
    mean_y = sum(y) / n
    numerator = sum((x-mean_x)*(y-mean_y))
    denom_x = sum((x-mean_x)**2)
    denom_y = sum((y-mean_y)**2)
    corr = numerator/sqrt(denom_x*denom_y)



    print *, "Correlation Coefficient: ", corr

end program Lab_23mar_26
~
~
~
~
~
~
"correl_1.f90" 23L, 534B                                    5,1          All
```

As we can see in this code , we have defined two arrays ; x = (13,55,84,36,58,29,12,25,97) and y = (71,9,26,70,19,50,30,23,22,44). So basically , firstly we have found out the mean of both x and y ( mean _x and mean_y)

- Next , we have determined the sum of ( y- mean_y) and (x-mean_x) , which will be the numerator
- Then , we have determined the sum of the square of ( y – mean_y) and sum of the square of (x – mean_x) ; and ,multiplied these values
- On multiplying these the above obtained values gives us our denominator
- At last , we will divide the denominator by numerator to obtain the correlation coefficient , which is coming out to be :

```
user@DESKTOP-Q2I4J5P:~$ ./a.out
 Correlation Coefficient:    -6.90870434E-02
```

**Q.3) Write a Fortran program to estimate the RMSE between two one dimensional arrays, one representing the observation and other from simulation.**

Solution – For computing the RMSE (Root mean square error) , we have used the following code :

```
program rmse_calc
    implicit none

    real, dimension(10) :: obs, sim
    real :: rmse , diff_sq_sum
    integer :: n

    obs = (/ 10,20,30,40,50,60,70,80,90,100 /)
    sim = (/ 12,24,33,41,56,64,78,82,95,105 /)
    n = size(obs)


        diff_sq_sum = sum((obs - sim)**2)
        rmse = sqrt(diff_sq_sum / n)


    print *, "Root Mean Square Error (RMSE): ", rmse

end program rmse_calc
"rmse.f90" 19L, 396B                                    5,30          All
```

As we can see that in this , we have two arrays , one representing the observed values , obs = ( 10,20,30,40,50,60,70,80,90,100 ) , and another one representing the simulation values , sim = ( 12,24,33,41,56,64,78,82,95,105 )

- Firstly , we will determine the size of arrays , since both have same size here , we can assume the size of any one array as n
- Next , we will sum up the squared values obtained from the observation values – simulation values .
- Also , we will the divide it with the size of array n , the take out the square root of it .

The obtained value is the RMSE , which is coming out to be :

```
user@DESKTOP-Q2I4J5P:~$ ./a.out
 Root Mean Square Error (RMSE):    4.47213602
```

**Q.4) Write a Fortran program to sort a one-dimensional array. Modify the program to read input data from a text/ascii file and write to a text/ascii file.**

Solution – For performing this operation in Fortran , we must be using ‘.txt’ file to perform our operations on ; instead of f.90 file .So , we will create and save

‘input.txt’ file to take input values from ; and ‘. Also the flow of the code should be like as follows :

```
user@DESKTOP-Q2I4J5P: ~

program read_write
        integer, dimension(5) :: a
        integer :: i,j,k
        open( 1, file = "input.txt")
        open(2, file = "output.txt")
        do i = 1,5
        read(1,*) a(i)
        enddo

        do j = 1,4
        do i = 1,5-j

        if (a(i)>a(i+1)) then
                k = a(i)
                a(i) = a(i+1)
               a(i+1) = k

        end if
        end do
        enddo

        write(2,*) a


end program read_write



"read_write.f90" 29L, 464B                                    2,28          All
```

- So firstly , we will specify the dimension of the array a (here , 5) and integers i,j,k
- Next , opening the file ‘input.txt’ and assigning it a number as 1 ; similarly opening the file ‘output.txt’ and assigning it number as 2
- Line no 6 to 8 makes sure to read the file 5 times
- Then ,do j = 1,4 makes sure to repeat the iterating process 4 times and do i =1,5-j makes sure the range of index number in which the sorting will happen .
- ‘If’ loop is the loop which actually sorts the numbers given in the ‘input.txt.’ file .
- “Write(2,*) a “ writes these values on the ‘output.txt’ file

So , initially the input.txt file contains value randomly ordered

```
12
34
5
67
89
-- INSERT --                                                    5,3          All
```

And after running the sort program , the output is :

```
5          12          34          67          89
```

**Q.5) Write a Fortran program to find the maximum and minimum of a one-dimensional array.**

Solution – To find maximum and minimum of an array , we will use the following code :

```
user@DESKTOP-Q2I4J5P: ~
program max_min_array
    implicit none

    real, dimension(10) :: arr
    real :: max_val, min_val
    integer :: n


    arr = (/ 40,55,22,34,76,88,93,12,73,90 /)
    n = size(arr)


    max_val = maxval(arr)
    min_val = minval(arr)

    print *, "Maximum value: ", max_val
    print *, "Minimum value: ", min_val

end program max_min_array
~
~
~
~
~
~
~
~
~
~
"max_min.f90" 19L, 346B                                    1,1          All
```

As for finding the max and min of the array , we will simply define an array , here it is arr = (40,55,22,34,76,88,93,12,73,90 ) . To find maximum of it , we will use “maxval(arr)” ; and to find minimum value , we will use “minval(arr)” . And then use the print function to obtain these values

```
user@DESKTOP-Q2I4J5P:~$ ./a.out
 Maximum value:     93.0000000
 Minimum value:     12.0000000
```

## PYTHON PROGRAMMING

**Q.1) Read the netcdf file provided, note down the names of all the variables, the corresponding long_name, and units. Also, note down the dimension names and sizes of the variables.**

Solution – The net cdf file provided has the following variables and dimensions :

```
<xarray.Dataset> Size: 5MB
Dimensions:  (COADSX: 180, COADSY: 90, TIME: 12)
Coordinates:
  * COADSX   (COADSX) float64 1kB 21.0 23.0 25.0 27.0 ... 375.0 377.0 379.0
  * COADSY   (COADSY) float64 720B -89.0 -87.0 -85.0 -83.0 ... 85.0 87.0 89.0
  * TIME     (TIME) float64 96B 366.0 1.096e+03 ... 7.671e+03 8.401e+03
Data variables:
    SST      (TIME, COADSY, COADSX) float32 778kB ...
    AIRT     (TIME, COADSY, COADSX) float32 778kB ...
    SPEH     (TIME, COADSY, COADSX) float32 778kB ...
    WSPD     (TIME, COADSY, COADSX) float32 778kB ...
    UWND     (TIME, COADSY, COADSX) float32 778kB ...
    VWND     (TIME, COADSY, COADSX) float32 778kB ...
    SLP      (TIME, COADSY, COADSX) float32 778kB ...
Attributes:
    history:  FERRET V4.45 (GUI) 22-May-97
```

**DATA VARIABLES**

- **COADSX** – Longitude values , ranging from 21°E to 379°E , having 180 steps with a resolution of 2°
- **COADSY**- Latitude values , ranging from -89°N to 89°N , having 90 steps with a resolution of 2°
- **TIME** – Time values , hourly data with the resolution not clearly visible

**DIMENSIONS**

- **SST** - SEA SURFACE TEMPERATURE (in °C)
- **AIRT** - AIR TEMPERATURE (in °C)
- **SPEH** - SPECIFIC HUMIDITY(in g/kg)
- **WSPD**- WIND SPEED (in m/s)
- **UWND** - ZONAL WIND ( in m/s)
- **VWND**- MERIDIONAL WIND (in m/s)
- **SLP** - SEA LEVEL PRESSURE ( in mb)

**Q2)Estimate the saturation vapor pressure, using the sea surface temperature provided in the netcdf file.**

Solution –

```
temp = ds['SST']
press = ds['SLP']

es = 6.112*np.exp((17.67*temp)/(243.5+temp))
```

Where , temp is the SST values  and Es is the saturation vapour pressure .

**Q3) Estimate the saturation specific humidity. Use the sea level pressure provided in the netcdf file and the saturation vapor pressure from above.**

Solution –

```
qs = (0.622*es)/(press - 0.378*es)
```

Where , qs represents the specific humidity , es represents the saturation vapour pressure ,and press represents pressure .